

Study Plan — Closing the Post-Training Gap

Created 2026-08-16. Owner: Monthir Ali. Status: not started.

Companion to [cv.md](#) and [modes/_profile.md](#). Update the **Claim ledger** (§6) as each phase lands — that is the only section the CV is allowed to draw from.

1. Why this, and why not the obvious version

Post-training is now the binding technical constraint in the search. Evidence from the last three batches:

Role	Score	What blocked it
Nuro — Applied AI Researcher, Agent Systems & Eval 286	4.5	SFT/RL on open-weight VLMs is half the charter
Waymo — ML Engineer, Foundation Model Recipes 292	3.7	<i>"prior work developing recipes for ML models"</i> — the only real requirement
LangChain — RE, LangSmith Engine 287	4.0	post-training in the responsibilities, RLHF/SFT/DPO in nice-to-have
Epic Games — MLE, Memory	killed at gate	<i>expert</i> hands-on post-training of open-weights models
Nuro — SWE, Applied AI Infrastructure	killed at gate	full post-training stack + inference internals

Unlike "0 industry years", this one can be closed by doing the work.

But do not build the obvious version. A repo called [llama-finetune](#) proves nothing — there are tens of thousands of them, and none of them establish whether the fine-tune actually helped. That version of this project makes you the hundred-thousandth person with a LoRA notebook.

Build the version only you can build. Nuro's own wording is the brief:

"Hands-on post-training experience — SFT and RL, ideally on open-weight models — **including the data curation and evaluation work required to know whether it actually helped.**"

The second half of that sentence is already yours. The eval harness, the cluster bootstrap, the power analysis, the pre-registration discipline, the judge-bias work — all of it exists and none of it has ever been pointed at a training run.

The thesis

Post-training changes are routinely accepted on evidence that cannot support them. Run real SFT and preference-optimization on open-weight models, then measure the results with the same inference machinery used to measure judges — and report honestly how much of the claimed improvement survives.

That is a question, not a tutorial. It closes the gap *and* extends your existing differentiator instead of competing on someone else's turf. The likely headline — "*a large fraction of reported post-training gains are inside the noise floor at the sample sizes people actually evaluate at*" — is exactly the kind of result you have already shown you will publish even when it is inconvenient.

2. Scope and shape

- **Time:** ~6–10 hrs/week alongside the postdoc. First claimable milestone at **week 2**; shippable public artifact at **week 6**; complete at **week 12**.
 - **Output:** extend the existing **eval-harness** repo with a **posttrain/** track, or a sibling repo that imports it. **Prefer extending** — it compounds the artifact you already have rather than splitting your evidence across two repos.
 - **Hard rule:** every phase ends with something you can state truthfully on the CV. If a phase slips, the earlier claims still stand.
-

3. Phases

Phase 1 — SFT mechanics (weeks 1–2) · *first claim unlocked*

Get the machinery working end to end on a small open-weight instruct model.

- Model: a current open-weight **1–4B** instruct model (Qwen3-1.7B, Llama-3.2-1B/3B, Gemma-3-1B/4B class — pick whatever is current when you start, and record the exact checkpoint). See §4 for why small is the right call, not a compromise.
- Method: **LoRA / QLoRA** via **peft** + **trl**'s **SFTTrainer**, with **unsloth** for speed and VRAM headroom on a free T4. **axolotl** if you want config-driven runs you can diff.
- Task: pick something *you* have data for and can score. Strong candidate — **convert natural-language experiment descriptions into SweetPea DSL programs**. You own the domain, you can generate training pairs from the SweetPea test suite and docs, and correctness is checkable by running the compiler. That last property is rare and valuable: it gives you a verifiable reward for Phase 5.
- Deliverables: a reproducible training config, a pinned dataset build script, loss curves, and a held-out set you did not touch.

Claim unlocked: *supervised fine-tuning of open-weight models (LoRA/QLoRA); training-data curation; reproducible training configs.*

Phase 2 — Preference optimization (weeks 3–4)

- Method: **DPO** first — simplest, most defensible, and the one named most often in the JDs. **trl**'s **DPOTrainer**. Add ORPO/SimPO only if time is free.
- Data: build preference pairs from Phase 1's task (compiler-passing vs compiler-failing outputs give you clean, non-arbitrary preferences), or use an established set (UltraFeedback / HH-RLHF) as a sanity baseline.
- Deliverables: base vs SFT vs DPO, three checkpoints, one fixed eval protocol.

Claim unlocked: *preference optimization (DPO) on open-weight models.*

Phase 3 — The differentiated part (weeks 5–6) · *ship here*

Point the eval harness at your own checkpoints. This is the phase that makes the project yours rather than generic.

- Treat the three checkpoints as a **paired comparison over prompt clusters** — exactly the data model the harness already assumes.
- Produce for every claimed improvement: a cluster-robust interval, a permutation p-value, the **noise floor**, and the **MDE at the sample size you actually ran**.
- Pre-register: declare in advance what counts as a real improvement, how many items you need, and the falsification criteria. Same discipline as the position-bias study.
- Run the honest cross-check: **evaluate the same checkpoints with a naive "run 50 examples and pick the winner" protocol**, and report how often that protocol picks a different winner than the defensible one. That comparison is the paper.
- Expect and report the winner's-curse effect you already quantified (a true 0.10 effect reading as 0.245 when you pick the best of four) — now on real checkpoints rather than in simulation.

Claim unlocked: *end-to-end post-training with statistically defensible evaluation — the full loop Nuro describes. Publish and update the CV at this point even if you go no further.*

Phase 4 — Close **agreement**/ (weeks 7–8)

The one unchecked box in the eval-harness README, and the obvious question from any annotation company — SuperAnnotate above all.

- Implement judge-vs-human calibration: Cohen's/Fleiss' kappa, **Krippendorff's alpha**, and agreement intervals consistent with the rest of the library's inference model.
- Collect a modest human-label set on your Phase 3 outputs. You have run N=100+ human studies; this is the part of the project you are *most* qualified to do and nobody else in the fine-tuning crowd can.
- Tie it back: does the LLM judge you used in Phase 3 agree with humans well enough to have carried the conclusion? Given your competence-gate finding, be prepared for the answer to be no — and to say so.

Claim unlocked: *judge-vs-human agreement calibration (kappa, Krippendorff's alpha). Until this ships, that claim stays off the CV — see §6.*

Phase 5 — RL (weeks 9–12) · *stretch, cut first*

- Method: **GRPO** via **trl**'s **GRPOTrainer** — currently the most tractable single-GPU RL path, and it needs a verifiable reward.
- Reward: if you took the SweetPea-DSL task in Phase 1, you already have one — does the generated program compile and satisfy the stated constraints. That is a clean, non-gameable signal, which is unusual and worth saying out loud.
- Deliverable: even a *negative* result here is publishable and in your voice — "GRPO did not beat DPO on this task at this scale, and here is the interval that shows it."

Claim unlocked: *reinforcement learning (GRPO) for post-training with verifiable rewards.*

4. Compute and budget — target \$0 (*revised 2026-08-16*)

The first draft of this plan assumed rented GPUs at \$150–400. **That is not necessary, and paying it while job-hunting would be a bad trade.** Run the whole thing on free tiers.

The free stack

- **Kaggle Notebooks — the workhorse.** ~30 GPU-hours per week, free, with sessions up to ~9–12 hours. Two T4s (16 GB each) or a P100. This is substantially more generous than Colab's free tier and is enough for every phase here.
- **Google Colab free tier** — a T4, shorter sessions, aggressive disconnects. Use as overflow, not as the primary.
- **Unsloth** — roughly 2× faster training and a large VRAM reduction versus a stock `trl` + `peft` loop. It is what makes QLoRA comfortable on a single T4. Use it from day one.

Shrink the model, not the rigour

Drop from 7–8B to a **1–4B open-weight instruct model** (Qwen3-1.7B, Llama-3.2-1B/3B, Gemma-3-1B/4B class — pick what is current). QLoRA on a 1–3B model trains comfortably on a free T4 in minutes-to-hours rather than hours-to-days.

This does not weaken the artifact, and it is important to understand why. The thesis is *methodological* — that post-training gains are routinely accepted on evidence that cannot support them. That claim is established by the quality of the inference, not by the size of the model. A 1.7B model with a pre-registered protocol, a stated noise floor, and honest intervals is a **better** artifact than an 8B model with a leaderboard number and no uncertainty. It is also faster to iterate on, which means more experiments, which is what the argument actually needs.

If anyone asks why 1.7B: *"because the question was whether the evaluation supports the claim, and model scale is orthogonal to that. I ran more seeds instead."* That is a strong answer, not an apology.

Eliminate the API bill too

Phase 3 does **not** need a paid LLM judge. Use the **compiler as the grader** — if the Phase 1 task is natural-language → SweetPea DSL, correctness is "does it compile and satisfy the stated constraints", which is free, deterministic, and *more* defensible than a judge. Keep judge-based scoring as an optional side-experiment only if you have credits lying around; the harness already caches responses and does dry-run cost accounting, so you would see the bill before spending it.

Revised budget

Item	Cost
Training compute (Phases 1–3, 5)	\$0 — Kaggle free GPU hours
Grading / evaluation	\$0 — compiler-verifiable correctness
Storage, model weights	\$0 — Hugging Face free tier
Total	\$0

Optional later, only if it is ever worth it to *you*: Colab Pro at ~\$10/month for longer sessions. Do not spend more than that until you are employed.

The constraint that actually binds

It is **time**, not money — ~6–10 hrs/week, and free-tier sessions disconnect. Two consequences to design around from the start: checkpoint to Hugging Face or Drive after every epoch so a dropped session costs minutes rather than a run, and keep every run seeded and config-driven so an interrupted job resumes instead of restarting. You already built resumable append-only records into the eval harness — same instinct, apply it here.

5. What to cut under pressure

In order: **Phase 5** → **Phase 4** → **Phase 2**. Never cut Phase 3 — without it this is a commodity fine-tuning repo and the whole strategic point is lost. If you only ever finish Phases 1 and 3, you still have a genuinely differentiated artifact and the two most-requested claims.

6. Claim ledger — the fabrication guard

Nothing from this project goes on the CV, into a cover letter, or into an interview answer until its phase is actually done and public. Tick the box here first; `cv.md` and `modes/_profile.md` may only draw from ticked rows.

Phase	Claim it authorises	Done?	Date
1	SFT (LoRA/QLoRA) on open-weight models; training-data curation	☐	
2	Preference optimization (DPO)	☐	
3	Post-training evaluated with cluster-robust inference, power, pre-registration	☐	
4	Judge-vs-human agreement calibration (kappa, Krippendorff's alpha)	☐	
5	RL (GRPO) with verifiable rewards	☐	

Currently on the CV and NOT authorised by anything: nothing — the standing `agreement/` guard in `modes/_custom.md` and `cv-variants/README.md` stays in force until Phase 4 ticks.

In the meantime, the honest interview line is unchanged and it is a good one:

"I came at models from the measurement side — I can tell you whether a post-training change actually worked, which is the part most teams get wrong. The training-recipe craft itself I'm building now; here's what I've shipped so far."

7. The prompt for the desktop Claude app

Paste this as the first message of a new project/conversation. It sets up a working collaborator rather than a tutorial generator. Keep the conversation going across sessions rather than starting fresh each time.

I'm running a self-directed engineering project and I want you as a hands-on collaborator on it – not a tutorial writer. Push back on my choices when you disagree, and tell me when something I'm proposing is a waste of time.

WHO I AM

PhD in Computer Science (Dec 2024), currently a postdoc research engineer. My background is human perception / XR research and, more recently, statistical evaluation of LLM systems. I built and published an open-source evaluation library (github.com/Mnznrabusham/eval-harness): cluster bootstrap and permutation inference, power/MDE analysis, judge-bias measurement, and a pre-registered study run against live judge models where the competence gate failed for all three judges and I published that rather than dropping the criterion.

I am strong at: experimental design, statistical inference, evaluation infrastructure, Python, reproducible pipelines, saying when a result doesn't hold. I am weak at: model training itself. I have never run SFT, DPO, or RL. My fine-tuning experience is API-level only. That gap is the reason for this project.

WHAT I'M BUILDING

Not "a fine-tuning project" – the world has enough of those. The thesis is:

Post-training changes are routinely accepted on evidence that cannot support them. I want to run real SFT and preference optimization on open-weight models, then evaluate the results with the same statistical machinery I use to evaluate judges, and report honestly how much of the claimed improvement survives contact with a proper noise floor.

Phases:

1. SFT mechanics (LoRA/QLoRA, trl + peft) on a 7-8B open-weight instruct model
2. Preference optimization (DPO)
3. Evaluate all checkpoints with my eval-harness: cluster-robust intervals, permutation tests, pre-registered acceptance criteria, MDE at the sample size I actually ran – plus a head-to-head against the naive "run 50 examples and pick the winner" protocol to show how often it picks a different winner
4. Build the judge-vs-human agreement module I never finished (kappa, Krippendorff's alpha)
5. Stretch: GRPO with a verifiable reward

Candidate task: fine-tune the model to translate natural-language experiment descriptions into SweetPea DSL programs (SweetPea is an open-source factorial experiment-design language I'm a maintainer of). Correctness is checkable by running the compiler, which gives me a verifiable reward for phase 5. Tell me if you think there's a better task – I'm not attached to it.

CONSTRAINTS

- ~6-10 hrs/week, and I am job-hunting, so my compute budget is effectively \$0. I plan to run entirely on Kaggle's free GPU tier (~30 hrs/week, T4/P100) using Unsloth, on a 1-4B open-weight instruct model rather than a 7-8B one, and to grade correctness with a compiler rather than a paid LLM judge.
- I believe shrinking the model does not weaken the argument, because the claim is about whether the evidence supports the conclusion, and that is orthogonal to model scale. Tell me if you think I'm wrong about that.
- Free-tier sessions disconnect, so everything must checkpoint and resume, and everything must be seeded and config-driven.
- I care more about being able to defend every number than about leaderboard wins.

HOW I WANT YOU TO WORK WITH ME

- Start by interrogating the plan. Where is it wrong, over-scoped, or naive about

how post-training actually behaves? What will bite me that I haven't anticipated?

- Then take Phase 1 only. Walk me through the decisions a practitioner actually makes – data formatting and templating, packing, LoRA rank/alpha/target modules, learning rate and schedule, batch size and gradient accumulation, when eval loss is lying to me – and explain the **reasoning**, not just the values. I want to understand the recipe space, not copy a config.
- Give me runnable code, but make me make the choices.
- Assume I'll paste back errors, loss curves, and results, and that we'll iterate.
- Be blunt about what a hiring manager would find unimpressive in what I produce.

Start by challenging the plan, then ask me whatever you need to know before we begin Phase 1.

Notes on using it

- **Keep one long conversation** (or a Claude Project) rather than starting fresh each session — the value compounds as it accumulates your actual results.
- **Paste real artifacts back:** loss curves, eval output, stack traces. Vague descriptions get vague help.
- When you reach Phase 3, tell it explicitly that the eval harness already exists and you are *importing* it, not rebuilding it — otherwise it will helpfully reimplement a bootstrap you already wrote and validated.
- Bring the results back here as each phase lands and I'll tick the ledger in §6 and update `cv.md` and `modes/_profile.md` from it.